

UT1 · Arquitectura del lado cliente

Módulo 0612 · Desarrollo Web en Entorno Cliente (DEW)

CFGS Desarrollo de Aplicaciones Web · 2.º curso · Curso 2026/27

Resultado de aprendizaje	RA1. Selecciona las arquitecturas y tecnologías de programación sobre clientes web, identificando y analizando las capacidades y características de cada una.
Duración	5 horas (semana 1)
Hito de la unidad	Comprendes qué vas a construir durante el curso y tienes un proyecto ejecutable bajo control de versiones.

Índice

Para generar el índice: clic derecho aquí → Actualizar campo (Word) o Actualizar índice (LibreOffice).

Qué vas a aprender

Al terminar esta unidad serás capaz de:

- Explicar qué parte de una aplicación web se ejecuta en el servidor y qué parte en el navegador, y por qué.
- Distinguir una aplicación multipágina (MPA) de una aplicación de página única (SPA).
- Describir qué hace un navegador cuando carga una página y qué puede y qué no puede hacer el código que ejecuta.
- Situar HTML, CSS, JavaScript y TypeScript dentro del desarrollo en el cliente.
- Valorar las ventajas y los inconvenientes de los lenguajes de guiones frente a la programación tradicional.
- Integrar JavaScript en una página HTML de la forma adecuada.
- Diferenciar una librería de un framework y reconocer los más usados.
- Preparar y verificar tu entorno de trabajo: VS Code, navegador con DevTools, Node.js, Git y GitHub.

Criterios de evaluación (RA1)

CE	Se evaluará si...
a	Has caracterizado y diferenciado los modelos de ejecución de código en el servidor y en el cliente web.
b	Has identificado las capacidades y mecanismos de ejecución de código de los navegadores web.
c	Has identificado y caracterizado los principales lenguajes relacionados con la programación de clientes web.
d	Has reconocido las particularidades de la programación de guiones y sus ventajas y desventajas sobre la programación tradicional.
e	Has verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes web.
f	Has reconocido y evaluado las herramientas de programación y prueba sobre clientes web.

1. Qué vamos a construir este curso

Este módulo no se aprueba memorizando sintaxis, sino construyendo. Durante el curso vas a desarrollar **tu propio proyecto**: una aplicación web sobre un tema que tú elijas. Empezará siendo una página con unas pocas líneas de JavaScript y terminará siendo una aplicación Angular que guarda datos, valida formularios y se comunica con un servidor.

Cada unidad añade una capa. Así, cuando llegue Angular, no será una caja negra: sabrás qué problema resuelve porque antes lo habrás resuelto tú a mano.

1.1. Requisitos comunes

El tema es libre, pero todos los proyectos deben cumplir los mismos requisitos mínimos. Así se garantiza que tu proyecto sirva para demostrar todos los resultados de aprendizaje del módulo.

Código	Requisito
R1	Gestiona una colección de elementos de un tema que te interese.
R2	La entidad principal tiene al menos 6 propiedades de, como mínimo, 4 tipos distintos (texto, número, fecha, booleano, categoría...).
R3	Existe al menos una segunda entidad relacionada con la principal (por ejemplo, <i>Receta</i> e <i>Ingrediente</i> , o <i>Ruta</i> y <i>Valoración</i>).
R4	Permite dar de alta, consultar, modificar y eliminar elementos.
R5	Ofrece un listado con búsqueda, al menos un filtro, ordenación y dos o más estadísticas calculadas.
R6	Incluye un formulario validado , con alguna expresión regular.
R7	Conserva información en el navegador entre ejecuciones (datos o preferencias).
R8	Consume un servicio web : una API pública relacionada con tu tema o una API simulada que te proporcionaremos.
R9	La versión final está hecha en Angular , con al menos 3 componentes, 1 servicio y 2 rutas.
R10	Vive en un repositorio de GitHub con commits frecuentes y un README actualizado.

Ejemplos de temas que funcionan bien: recetario con ingredientes, colección de videojuegos o vinilos, diario de rutas de senderismo, control de gastos personales, reservas de pistas de un club, cuidado de plantas, biblioteca personal, inventario de material de un grupo de teatro, seguimiento de entrenamientos.

Importante. No uses datos personales reales de otras personas, no copies un tutorial completo y no elijas un tema que dependa de pagos o de inicios de sesión reales. El tema debe aprobarlo tu profesor o profesora; si dos propuestas son casi idénticas, se os pedirá que las diferenciéis.

1.2. Hoja de ruta

UT	Semanas	Lo que añades a tu proyecto
UT1	1	Ficha del proyecto, repositorio en GitHub y página inicial con JavaScript integrado.

UT	Semanas	Lo que añades a tu proyecto
UT2	2–4	Lógica básica en JavaScript sobre los datos de tu tema (cálculos, decisiones, bucles), probada en la consola.
UT3	5–6	Datos de ejemplo en JSON, formato de fechas y números, y persistencia en el navegador.
UT4	7–9	Funciones y colecciones: búsquedas, filtros, ordenaciones y estadísticas con operaciones agregadas.
UT5	10–12	Modelado con objetos, clases y módulos; primeros tipos en TypeScript; primer componente Angular.
UT6	13–15	Interfaz generada a partir de los datos: primero manipulando el DOM y después con plantillas de Angular.
UT7	16–18	Interacción: eventos, formulario de alta y edición con validación, varios componentes.
UT8	19–20	Comunicación con un servicio web, con estados de carga, vacío y error.
UT9	21–22	Aplicación Angular completa: servicios, HttpClient, rutas y despliegue.

2. La aplicación web: el modelo cliente-servidor

2.1. Cliente, servidor y protocolo ★

Una aplicación web se reparte entre dos programas que se comunican a través de la red:

- **El cliente:** normalmente, un navegador web (Chrome, Firefox, Safari, Edge...). Pide recursos, los interpreta y los muestra.
- **El servidor:** un programa que espera peticiones y devuelve respuestas. Puede entregar ficheros tal cual o generarlos en ese momento a partir de una base de datos.
- **El protocolo:** el idioma común. En la web es **HTTP** (o **HTTPS**, su versión cifrada). Funciona siempre igual: el cliente envía una **petición** y el servidor contesta con una **respuesta**.

Nota. HTTP es un protocolo **sin estado**: cada petición es independiente y el servidor no «recuerda» al cliente entre una y otra. Para mantener sesiones o preferencias existen mecanismos como las *cookies* o el almacenamiento del navegador, que estudiarás en la UT3.

3. Ejecución de código en el servidor y en el cliente

La pregunta clave de esta unidad es **dónde se ejecuta cada parte del código**.

3.1. Código en el servidor ★

El código de servidor (PHP, Java, Python, C#, JavaScript con Node.js...) se ejecuta **antes de enviar la respuesta**. Consulta la base de datos, aplica las reglas del negocio y construye el resultado. El navegador solo recibe el producto final: HTML, JSON, una imagen... **Nunca ve el código fuente**. Este modelo lo trabajas en el módulo de Desarrollo Web en Entorno Servidor.

3.2. Código en el cliente ★

El código de cliente se **descarga junto con la página y se ejecuta en el navegador**, en el dispositivo del usuario. Hoy ese lenguaje es, en la práctica, **JavaScript**. Permite reaccionar al instante a lo que hace el usuario (un clic, una tecla), modificar la página sin recargarla, validar un formulario antes de enviarlo o pedir datos al servidor en segundo plano.

3.3. Comparativa ★

Aspecto	Código en el servidor	Código en el cliente
Dónde se ejecuta	En la máquina del servidor	En el navegador del usuario
Cuándo	Al recibir una petición, antes de responder	Tras descargar la página, y mientras el usuario la usa
¿Se ve el código?	No	Sí: cualquiera puede leerlo (DevTools, Ctrl+U)
Acceso a datos	Bases de datos, ficheros y servicios internos	Solo lo que el servidor le envíe y lo que guarde el propio navegador
Seguridad	Zona de confianza	Zona no confiable: el usuario puede alterar el código
Rendimiento	Depende del servidor y del número de usuarios	Depende del dispositivo del usuario
Lenguajes	Muchos (PHP, Java, Python, C#, Node.js...)	JavaScript (y lo que se traduce a él o a WebAssembly)

Aspecto	Código en el servidor	Código en el cliente
Usos típicos	Autenticación, reglas del negocio, persistencia central	Interactividad, validación inmediata, interfaces dinámicas

3.4. No compiten: colaboran

En una aplicación real ambos lados trabajan juntos. El ejemplo más claro es la validación de un formulario:

- **En el cliente**, se valida para dar una **buena experiencia de usuario**: el aviso aparece al momento, sin esperar al servidor.
- **En el servidor**, se valida por **seguridad**, porque la validación del cliente se puede saltar: basta con desactivar JavaScript, modificar el código desde las DevTools o enviar la petición con otra herramienta.

Importante. Regla de oro del desarrollo web: **nunca confíes en el cliente**. Todo lo que llega del navegador puede haber sido manipulado. Las contraseñas, las claves de API privadas o los cálculos de precios definitivos no deben depender solo del código de cliente.

3.5. Modelos de renderizado

Según quién construya el HTML que ve el usuario, se habla de:

- **Renderizado en el servidor (SSR)**: el servidor envía la página ya construida. Es el modelo clásico (PHP, JSP...).
- **Renderizado en el cliente (CSR)**: el servidor envía un HTML casi vacío y un programa JavaScript que construye la interfaz en el navegador. Es el modelo de las SPA.
- **Modelos híbridos**: el servidor envía una primera versión ya renderizada (buena para la carga inicial y el posicionamiento en buscadores) y después JavaScript «toma el control» de la página, un proceso llamado **hidratación**. Otra variante es la **generación estática (SSG)**, que construye las páginas al publicar el proyecto.

Angular permite los tres enfoques. En este curso trabajaremos con **renderizado en el cliente**, que es el que mejor muestra lo que ocurre en el navegador.

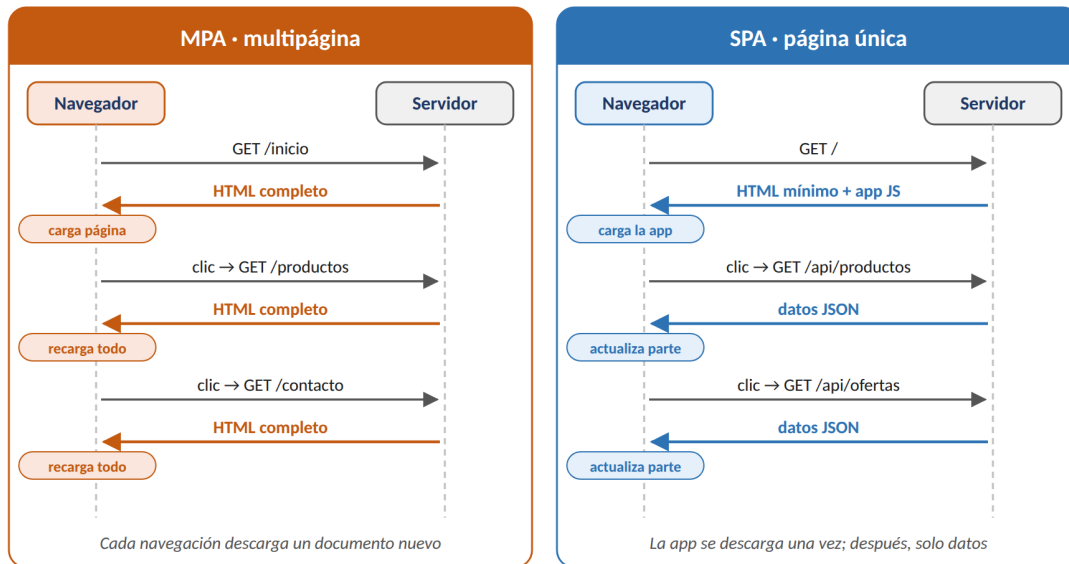
4. Aplicaciones multipágina (MPA) y de página única (SPA)

4.1. MPA: multipágina ★

En una **MPA** (*Multi-Page Application*), cada vez que el usuario navega a otra sección, el navegador pide al servidor un **documento HTML nuevo** y la pantalla se recarga por completo. Es el modelo original de la web y el de muchos sitios de contenido: Wikipedia, periódicos, blogs, webs institucionales.

4.2. SPA: página única ★

En una **SPA** (*Single-Page Application*), el navegador descarga **una sola vez** un HTML mínimo y la aplicación JavaScript. A partir de ahí, al navegar **no se recarga la página**: JavaScript cambia la parte de la interfaz que corresponde y, si necesita datos, los pide al servidor en segundo plano, normalmente en formato JSON. La URL también cambia, pero la gestiona la propia aplicación (**enrutado en el cliente**). Ejemplos: Gmail, Google Maps, las aplicaciones web de Spotify o de Trello... y lo que construirás con Angular.



MPA frente a SPA: en la MPA cada navegación descarga un documento nuevo; en la SPA se descarga la aplicación una vez y después solo se intercambian datos.

4.3. Comparativa ★

Aspecto	MPA	SPA
Navegación	Recarga completa en cada página	Sin recargas; cambia solo lo necesario
Carga inicial	Rápida (una página)	Más lenta (hay que descargar la aplicación)
Navegaciones siguientes	Más lentas (documento completo)	Muy rápidas (solo datos)
Sensación de uso	Web «clásica»	Parecida a una aplicación de escritorio o móvil
Posicionamiento en buscadores	Sencillo	Requiere técnicas adicionales (SSR, prerenderizado)
Complejidad	Menor en el cliente	Mayor: el cliente gestiona estado, rutas y errores

Aspecto	MPA	SPA
Funciona sin JavaScript	Normalmente sí, al menos en parte	Normalmente no

No hay un modelo «mejor»: se elige según el tipo de proyecto. Un periódico prioriza la carga inicial y los buscadores; una herramienta de gestión que se usa durante horas prioriza la fluidez.

Consejo. Para saber si una web es MPA o SPA, abre las DevTools, pestaña **Network**, filtro **Doc**, y navega por el sitio. Si cada clic genera una petición de tipo documento, es una MPA. Si solo aparecen peticiones de datos (**Fetch/XHR**), es una SPA. Lo pondrás en práctica en la actividad A2.

5. El navegador como entorno de ejecución

Un navegador moderno es mucho más que un visor de páginas: es una **plataforma de ejecución de aplicaciones**, con su propio motor de programación, su almacenamiento, su gestión de red y sus reglas de seguridad.

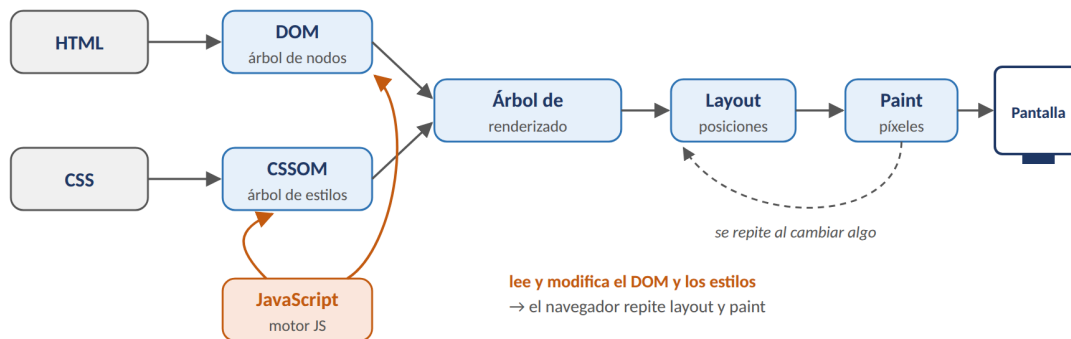
5.1. Componentes principales ★

- **Interfaz de usuario:** barra de direcciones, pestañas, botones...
- **Motor de renderizado:** interpreta HTML y CSS y dibuja la página.
- **Motor de JavaScript:** ejecuta el código JavaScript.
- **Red:** gestiona las peticiones HTTP, la caché y las conexiones.
- **Almacenamiento:** *cookies*, *localStorage*, *sessionStorage*, IndexedDB, caché...

Navegador	Motor de renderizado	Motor de JavaScript
Chrome, Edge, Opera, Brave	Blink	V8
Firefox	Gecko	SpiderMonkey
Safari	WebKit	JavaScriptCore

El motor V8 también es la base de **Node.js**, que permite ejecutar JavaScript fuera del navegador y que usaremos como herramienta de desarrollo.

5.2. Cómo se procesa una página ★



Del HTML a los píxeles: el navegador construye el DOM y el CSSOM, los combina en el árbol de renderizado, calcula la disposición y pinta. JavaScript puede modificar el DOM y el CSSOM y provocar que el proceso se repita.

1. El **analizador de HTML** lee el documento y construye el **DOM** (*Document Object Model*), un árbol de objetos que representa la página. Lo estudiarás más a fondo en la UT6.
2. El CSS se analiza y se construye el **CSSOM**.
3. DOM y CSSOM se combinan en el **árbol de renderizado**, que contiene solo lo visible.
4. Se calcula la posición y el tamaño de cada elemento (**layout**).
5. Se **pinta** el resultado en pantalla.

JavaScript puede leer y modificar el DOM y los estilos en cualquier momento. Cada cambio visible obliga al navegador a repetir parte del proceso. Además, cuando el analizador encuentra una etiqueta `<script>` normal, **se detiene** hasta descargar y ejecutar ese código. Por eso importa dónde y cómo se incluye JavaScript (apartado 8).

5.3. Cómo se ejecuta JavaScript ★

- Los motores actuales no se limitan a interpretar: usan **compilación just-in-time (JIT)**. Empiezan interpretando y compilan a código máquina las partes que más se repiten.
- JavaScript se ejecuta en **un único hilo principal**, el mismo que se encarga de pintar la página. Si un script tarda mucho, la página se «congela».
- Para no bloquear, las operaciones lentas (peticiones de red, temporizadores, eventos del usuario) son **asíncronas**: se inician, el programa sigue y, cuando terminan, su resultado se procesa mediante el **bucle de eventos** (*event loop*). Lo trabajarás en detalle en la UT8.

5.4. Capacidades del navegador ★

Además del lenguaje, el navegador ofrece a JavaScript un gran número de interfaces, llamadas **Web APIs**:

API	Qué permite	Unidad
DOM	Leer y modificar la estructura y el contenido de la página	UT6
Eventos	Reaccionar a clics, teclas, envíos de formularios...	UT7
BOM (<code>window</code> , <code>location</code> , <code>history</code> , <code>navigator</code>)	Interactuar con la ventana, la URL y el historial	UT3
Almacenamiento web (<code>localStorage</code> , <code>sessionStorage</code> , <code>cookies</code>)	Guardar información en el dispositivo	UT3
Temporizadores (<code>setTimeout</code> , <code>setInterval</code>)	Programar tareas en el tiempo	UT3
<code>fetch</code>	Comunicarse con servidores sin recargar la página	UT8
Otras	Geolocalización, notificaciones, portapapeles, canvas, audio y vídeo, <code>workers</code> ...	—

5.5. Limitaciones y seguridad ★

El código de cliente procede de cualquier web, así que el navegador lo ejecuta en un entorno aislado (*sandbox*) con restricciones importantes:

- **No accede libremente al sistema de ficheros** ni a otros programas del equipo. Solo puede leer los ficheros que el usuario elija de forma explícita.
- **Política del mismo origen** (*same-origin policy*): un script solo puede leer respuestas de su mismo origen (esquema + dominio + puerto), salvo que el otro servidor lo autorice mediante **CORS**. Te encontrarás con esto en la UT8.
- **Permisos**: la cámara, la ubicación o las notificaciones requieren el consentimiento del usuario y, en general, una conexión HTTPS.
- **El código es visible y modificable** por el usuario.
- **Puede no ejecutarse**: el usuario puede desactivar JavaScript, un bloqueador puede impedir un script o puede fallar la red.
- **Depende del dispositivo**: un móvil de gama baja no tiene la potencia de tu ordenador de clase.

5.6. Compatibilidad y estándares

Hay muchos navegadores, versiones y dispositivos. Para que el mismo código funcione en todos existen estándares:

- **ECMAScript**: la especificación del lenguaje JavaScript. La mantiene el comité TC39 de Ecma International y publica una edición cada año (ES2015, ES2016... ES2026).
- **WHATWG y W3C**: estandarizan HTML, el DOM, `fetch`, CSS y el resto de las Web APIs.

Importante: Antes de usar una característica moderna, comprueba su compatibilidad en las tablas de **MDN Web Docs**, en **Can I Use** (caniuse.com) o con la etiqueta **Baseline**, que indica si una característica está disponible en todos los navegadores principales. Cuando hay que dar soporte a navegadores antiguos se usan **polyfills** (código que añade una funcionalidad que falta) y **transpiladores** (herramientas que convierten código moderno en código equivalente más antiguo).

A continuación puedes ver un ejemplo de la web Can I Use con el módulo *grid* de CSS:

Chrome	Edge [*]	Safari	Firefox	Opera	IE
4-28			2-39		
¹ 29-56			³ 40-51	10-27	
⁴ 57	² 12-15	3.1-10	⁴ 52-53	¹ 28-43	6-9
58-152	16-151	10.1-26.6	54-155	44-134	² 10
153	152	27	156	135	² 11
154-156		27.1-TP	157-159		

Vemos que *grid* funciona perfectamente (recuadro verde) en **Chrome** desde la versión 58 en adelante, en **Edge** desde la 16, en **Safari** desde la 10.1, en **Firefox** desde la 54 y en **Opera** desde la 44. Los recuadros verdes con una etiqueta indican compatibilidad con **errores** conocidos, los recuadros amarillos indican **compatibilidad parcial** y los recuadros rojos advierten de **incompatibilidad**.

6. Lenguajes y tecnologías del entorno cliente

6.1. Las tres capas de una página ★

Capa	Tecnología	Responsabilidad
Contenido y estructura	HTML	Qué hay en la página y qué significa cada parte
Presentación	CSS	Cómo se ve: colores, tipografía, disposición, adaptación a pantallas
Comportamiento	JavaScript	Qué hace: interacción, cálculos, comunicación con el servidor

Mantener las tres capas separadas (cada una en su fichero y sin mezclar responsabilidades) facilita el mantenimiento, la reutilización y el trabajo en equipo. Es un criterio que se evalúa en la UT6.

6.2. JavaScript

Breve historia

Año	Hito
1995	Brendan Eich crea en Netscape, en unos diez días, el lenguaje que acabaría llamándose JavaScript.
1997	Se publica el estándar ECMAScript (ECMA-262) para unificar las implementaciones.
2005	Se populariza AJAX: páginas que piden datos sin recargarse (Gmail, Google Maps).
2006	Aparece jQuery, la librería que simplificó el trabajo con el DOM durante una década.
2008	Google lanza Chrome con el motor V8, mucho más rápido.
2009	Llegan ES5 y Node.js: JavaScript sale del navegador.
2010–2014	Nacen AngularJS (2010), React (2013) y Vue (2014).
2015	ES6 (ES2015) moderniza el lenguaje: let , const , clases, módulos, funciones flecha, promesas.
2016	Se publica Angular (a partir de la versión 2), reescrito por completo y basado en TypeScript.
Desde 2016	Una edición nueva de ECMAScript cada año.

Características principales

- **Interpretado y compilado just-in-time:** no hay un paso de compilación que hagas tú; el motor se encarga.
- **Tipado dinámico:** las variables no tienen tipo, los valores sí. Una misma variable puede guardar un número y después un texto.
- **Tipado débil:** realiza conversiones automáticas entre tipos, a veces sorprendentes (**'5' * 2** vale **10**, pero **'5' + 2** vale **'52'**). Lo estudiarás en la UT2.
- **Multiparadigma:** admite programación imperativa, orientada a objetos (basada en prototipos, con sintaxis de clases) y funcional.
- **Las funciones son valores:** se pueden guardar en variables, pasar como argumentos y devolver.
- **Asíncrono y dirigido por eventos:** el código reacciona a lo que ocurre (clics, respuestas del servidor).

Nota. JavaScript y Java **no tienen relación**, más allá del nombre y de cierto parecido en la sintaxis. El nombre fue una decisión comercial de 1995.

6.3. TypeScript

TypeScript es un lenguaje creado por Microsoft en 2012 que **amplía JavaScript con tipos estáticos**. Todo código JavaScript válido es, en principio, TypeScript válido. El navegador no entiende TypeScript: antes de ejecutarlo hay que **transpilarlo** (traducirlo) a JavaScript.

```
// JavaScript: el error aparece al ejecutar (o no aparece nunca)
function precioConIgc(precio) {
  return precio * 1.07; // IGIC general del 7 %
}
precioConIgc('veinte'); // NaN, sin ningún aviso previo

// TypeScript: el error se detecta mientras escribes
function precioConIgc(precio: number): number {
  return precio * 1.07;
}
precioConIgc('veinte'); // Error: 'string' no es assignable a 'number'
```

Angular está escrito en TypeScript y lo utiliza en sus proyectos. **En este curso aprenderemos primero JavaScript**, porque es lo que realmente ejecuta el navegador, y en la UT5 daremos el paso a TypeScript.

6.4. Otras tecnologías del cliente

- **JSON**: formato de texto para intercambiar datos, basado en la sintaxis de objetos de JavaScript. Es el formato habitual entre cliente y servidor (UT3 y UT8).
- **WebAssembly (Wasm)**: formato binario que permite ejecutar en el navegador, a velocidad casi nativa, código escrito en C, C++, Rust u otros lenguajes. Se usa en editores de vídeo, juegos o aplicaciones de diseño. Complementa a JavaScript, no lo sustituye.
- **SVG y Canvas**: gráficos vectoriales y dibujo desde código.

Tecnologías desaparecidas. Durante años existieron alternativas que necesitaban **complementos (plugins)** instalados en el navegador o que solo funcionaban en uno concreto: los *applets* de Java, Adobe Flash, ActiveX o VBScript (este último, exclusivo de Internet Explorer). Desaparecieron por sus problemas de seguridad, porque no funcionaban en los móviles y porque dependían de una sola empresa. La lección es clara: en el cliente triunfan los **estándares abiertos** que todos los navegadores implementan.

7. Programación de guiones frente a programación tradicional

7.1. Qué es un lenguaje de guiones ★

Un **lenguaje de guiones (scripting language)** es un lenguaje pensado para ejecutarse **dentro de otro programa, el anfitrión**, y controlarlo o automatizarlo, sin un paso de compilación previo por parte de quien programa. En el caso de JavaScript, el anfitrión es el navegador (o Node.js). Otros ejemplos son Bash, Python usado como lenguaje de automatización o los scripts de las hojas de cálculo.

7.2. Comparación con lo que ya conoces ★

Probablemente el curso pasado estuviste programando principalmente en Java o en .Net. Comparemos un mismo programa:

```
// Java: se compila (javac) a bytecode y se ejecuta en la JVM
public class Saludo {
    public static void main(String[] args) {
        String nombre = "Ada";
        int edad = 36;
        System.out.println("Hola, " + nombre + ". Tienes " + edad + " años.");
    }
}

// C#: se compila a CIL y se ejecuta sobre .NET
using System;
public class Saludo
{
    public static void Main(string[] args)
    {
        string nombre = "Ada";
        int edad = 36;

        // Concatenando strings literales y variables con el operador +
        Console.WriteLine("Hola, " + nombre + ". Tienes " + edad + " años.");

        // Concatenando strings literales y variables mediante interpolación
        Console.WriteLine($"Hola, {nombre}. Tienes {edad} años.");
    }
}

// JavaScript: el navegador lo ejecuta directamente
const nombre = 'Ada';
const edad = 36;
console.log("Hola, " + nombre + " Tienes " + edad + " años."); // operador +
console.log(`Hola, ${nombre}. Tienes ${edad} años.`); // interpolación
```

Aspecto	Programación tradicional (Java)	Guiones (JavaScript)
Traducción	Compilación previa explícita	La hace el motor al ejecutar (JIT)
Tipos	Estáticos: se comprueban al compilar	Dinámicos: se comprueban al ejecutar
Estructura mínima	Clase y método <code>main</code>	Una simple línea ya es un programa
Entorno de ejecución	Un programa autónomo sobre la JVM	Dentro de un anfitrión (el navegador)
Detección de errores	Muchos, antes de ejecutar	La mayoría, al ejecutar

Aspecto	Programación tradicional (Java)	Guiones (JavaScript)
Distribución	Se entrega el programa compilado	Se entrega el código fuente

7.3. Ventajas y desventajas

Ventajas	Desventajas
Ciclo de desarrollo muy rápido: editas, recargas y pruebas.	Muchos errores solo aparecen al ejecutar, y a veces en casos concretos.
Menos código para tareas sencillas.	El tipado débil puede provocar conversiones de tipo inesperadas.
No hay que instalar nada en el cliente: basta un navegador.	El código fuente es visible (se puede minificar, pero no ocultar).
Multiplataforma: funciona en cualquier sistema con navegador.	Depende del anfitrión: versión del navegador, JavaScript desactivado...
Integración directa con el documento y con el usuario.	En proyectos grandes, sin disciplina, el código se vuelve difícil de mantener.

7.4. Una frontera cada vez más difusa

Hoy la distinción no es tan nítida: los motores compilan JavaScript a código máquina, TypeScript añade comprobación de tipos antes de ejecutar y las herramientas de construcción (Vite, Angular CLI) procesan, optimizan y empaquetan el código antes de publicarlo. En el desarrollo profesional, JavaScript se trata con el mismo rigor que cualquier otro lenguaje: control de versiones, analizadores de código, pruebas y documentación.

8. Integración de JavaScript en HTML

8.1. Tres formas de incluir código ★

En línea, dentro de un atributo de evento. Funciona, pero **mezcla contenido y comportamiento**, así que se desaconseja:

```
<button onclick="alert('¡Hola!')">Saludar</button>
```

Interno, dentro de una etiqueta `<script>` en el propio documento. Sirve para pruebas rápidas:

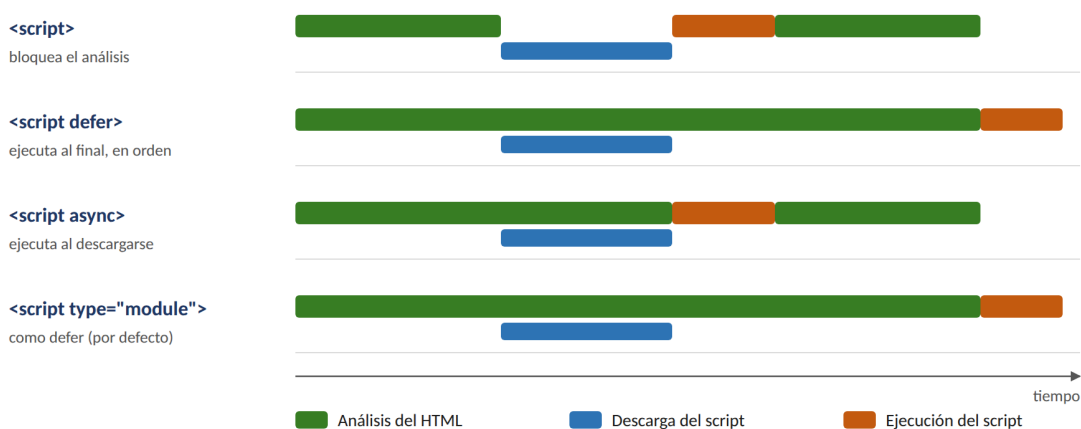
```
<script>
  console.log('Hola desde un script interno');
</script>
```

Externo, en un fichero `.js` enlazado. Es la **forma recomendada**: separa capas, se puede reutilizar en varias páginas y el navegador puede guardarlo en caché.

```
<script src="js/app.js" defer></script>
```

8.2. Cómo se carga un script: `defer`, `async` y `type="module"` ★

Por defecto, cuando el analizador de HTML encuentra un `<script>`, **se detiene**: descarga el fichero, lo ejecuta y solo entonces continúa. Si el script está en el `<head>` e intenta acceder a un elemento del `<body>`, ese elemento **todavía no existe**, por lo que es habitual situarlo al final del archivo, justo antes del cierre `</body>`.



Carga de scripts: el script normal detiene el análisis del HTML; defer descarga en paralelo y ejecuta al final, en orden; async ejecuta en cuanto termina la descarga; los módulos se comportan como defer.

Forma	Descarga	Ejecución	Orden entre scripts	Uso recomendado
<code><script src></code>	Bloquea el análisis	Inmediata	Sí	Evitar en el <code><head></code>
<code><script src defer></code>	En paralelo	Cuando termina el análisis del HTML	Sí	Scripts de la propia página
<code><script src async></code>	En paralelo	En cuanto se descarga	No garantizado	Scripts independientes (analítica, publicidad)
<code><script type="module" src></code>	En paralelo	Como <code>defer</code>	Sí	Código moderno con <code>import/export</code>

Los **módulos** (`type="module"`) tienen además estas particularidades: se ejecutan en modo estricto, sus variables no son globales, permiten `import` y `export` (UT5) y **necesitan un servidor HTTP**. Si abres la

página con doble clic (`file://`), el navegador los bloquea por la política de seguridad que viste en el apartado 5.5.

Recuerda. La opción habitual hoy es un script externo en el `<head>` con `defer`, o con `type="module"` si usas módulos. La costumbre antigua de poner los scripts al final del `<body>` también funciona, pero retrasa el comienzo de la descarga.

8.3. La etiqueta `<noscript>` ★

Su contenido solo se muestra si JavaScript está desactivado. Sirve para avisar al usuario de que la página lo necesita:

```
<noscript>Esta aplicación necesita JavaScript. Actívalo en tu navegador.</noscript>
```

8.4. Tu primera página con JavaScript ★

Estructura del proyecto:

```
hola-cliente/
├── index.html
├── css/
│   └── estilos.css
└── js/
    └── app.js
```

`index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hola, cliente</title>
  <link rel="stylesheet" href="css/estilos.css">
  <script src="js/app.js" defer></script>
</head>
<body>
  <h1>Hola, cliente</h1>
  <p id="info">Si ves este texto, JavaScript no se ha ejecutado.</p>
  <noscript>Esta página necesita JavaScript para funcionar.</noscript>
</body>
</html>
```

`js/app.js`:

```
// Este código se ejecuta en TU navegador, no en el servidor.
const info = document.querySelector('#info');
const ahora = new Date().toLocaleString('es-ES');

info.textContent = `Página actualizada en tu navegador el ${ahora}`;

console.log('Navegador:', navigator.userAgent);
console.log('Dirección:', location.href);
```

Todavía no hace falta que entiendas cada línea; lo irás aprendiendo en las próximas unidades. Fíjate en tres ideas:

1. `document` representa la página (el DOM) y `querySelector` localiza en ella un elemento.
2. `navigator` y `location` son objetos que el navegador pone a disposición del código (BOM).
3. `console.log` escribe en la **consola** de las DevTools, que será tu mejor aliada para depurar.

8.5. ¿Por qué no abrir la página con doble clic?

Si abres `index.html` con doble clic, la dirección empieza por `file://`. Así funcionan las páginas sencillas, pero no los módulos, ni las peticiones a ficheros JSON, ni otras funciones que exigen un origen web. Por eso, desde el primer día, trabajaremos con un **servidor de desarrollo local** (la extensión Live Server de VS Code), que sirve la página en una dirección como `http://127.0.0.1:5500` y además **recarga automáticamente** el navegador cada vez que guardas.

9. Librerías y frameworks

9.1. Qué los diferencia

Tanto las librerías como los frameworks son código escrito por otras personas que reutilizas para no reinventar la rueda. La diferencia está en **quién controla el flujo del programa**:

- Una **librería** es un conjunto de utilidades que **llamas** cuando las necesitas. Tu código manda. Ejemplos: jQuery, Lodash, Chart.js, date-fns. Su nombre correcto es **biblioteca**, pero se ha popularizado *librería* por transcripción del inglés *library*.
- Un **framework** es un marco de trabajo que define la estructura de la aplicación y **llama a tu código** en el momento adecuado. Tú rellenas las piezas que el framework espera. Este principio se llama **inversión de control**.

Una analogía: si quisiéramos construir una casa, una librería es un conjunto de herramientas con el que tú tienes que hacer todo, mientras que un framework es una casa prefabricada en la que tú solo tienes que ocuparte de decidir la distribución y la decoración.

9.2. Qué problema resuelven

En una aplicación interactiva, lo más difícil no es pintar la interfaz, sino **mantenerla sincronizada con los datos**: cuando cambia un dato, todas las partes de la pantalla que dependen de él deben actualizarse, y solo ellas. Hacerlo a mano con el DOM es laborioso y propenso a errores (lo comprobarás en la UT6). Los frameworks modernos resuelven ese problema: tú describes **cómo debe verse la interfaz según el estado** y el framework se encarga de actualizarla. Además, imponen una organización del código (componentes, servicios, rutas) que facilita el trabajo en equipo.

9.3. Angular, React y Vue en una tabla

	Angular	React	Vue
Qué es	Framework completo	Librería para construir interfaces	Framework progresivo
Creado por	Google (2016)	Meta (2013)	Evan You y comunidad (2014)
Lenguaje	TypeScript	JavaScript con JSX (o TypeScript)	JavaScript o TypeScript
Enfoque	«Todo incluido»: rutas, formularios, HTTP, pruebas...	Mínimo: el resto se añade con otras librerías	Intermedio: núcleo pequeño con piezas oficiales
Estructura	Muy definida, con convenciones fuertes	Libre: la decide cada equipo	Definida, pero flexible
Curva de aprendizaje	Más pronunciada al principio	Suave al principio, compleja al crecer	Suave
Uso típico	Aplicaciones empresariales grandes y de larga vida	Todo tipo de proyectos; ecosistema enorme	Proyectos pequeños y medianos; también grandes

9.4. ¿Por qué Angular en este curso?

- Es un **framework completo**: aprenderás una arquitectura entera (componentes, servicios, inyección de dependencias, rutas, formularios, HTTP), no solo una librería de vistas.
- Usa **TypeScript**, muy demandado en la industria.
- Tiene **convenciones claras**, lo que ayuda a aprender buenas prácticas y a leer código ajeno.
- Es muy utilizado en **empresas y administraciones**, justo el tipo de entorno en el que harás tu formación en empresa.
- Los conceptos que aprenderás (componentes, estado, renderizado declarativo, reactividad) se trasladan con facilidad a React o Vue.

Nota. Además de estos tres, existen otros como Svelte o Solid y los llamados **meta-frameworks** (Next.js, Nuxt, Analog), que añaden renderizado en el servidor y rutas basadas en archivos. El ecosistema cambia deprisa; los fundamentos de JavaScript, no.

10. Herramientas de programación y prueba

Herramienta	Categoría	Para qué la usaremos	Desde
Visual Studio Code	Editor de código	Escribir, depurar y gestionar el proyecto	UT1
Navegador + DevTools	Ejecución y depuración	Consola, inspección del DOM, red, almacenamiento, depurador	UT1
Live Server	Servidor de desarrollo	Servir la página en local con recarga automática	UT1
Git	Control de versiones	Guardar la historia del proyecto	UT1
GitHub	Repositorio remoto	Copia en la nube, entrega y portfolio	UT1
Node.js y npm	Entorno de ejecución y gestor de paquetes	Ejecutar herramientas de desarrollo e instalar dependencias	UT1 (instalación)
ESLint y Prettier	Calidad de código	Detectar errores y aplicar un formato uniforme	UT2
TypeScript	Lenguaje	Tipado estático	UT5
Angular CLI	Herramienta de línea de comandos	Crear, generar, servir, probar y construir proyectos Angular	UT5
Angular DevTools	Extensión del navegador	Inspeccionar componentes y su estado	UT5
Vitest	Pruebas	Pruebas unitarias en proyectos Angular	UT9

10.1. El editor: Visual Studio Code

VS Code es un editor gratuito y multiplataforma de Microsoft, el más usado en desarrollo web. Integra terminal, depurador, control de versiones y un enorme catálogo de extensiones. Atajos que usarás a diario:

Acción	Windows y Linux	macOS
Paleta de comandos	Ctrl + Mayús + P	Cmd + Mayús + P
Abrir o cerrar la terminal integrada	Ctrl + ñ (teclado español) o Ctrl + `	Ctrl + `
Buscar un fichero por nombre	Ctrl + P	Cmd + P

Acción	Windows y Linux	macOS
Formatear el documento	Mayús + Alt + F (en Linux, Ctrl + Mayús + I)	Mayús + Opción + F
Comentar o descomentar líneas	Ctrl + ¢ (teclado de España) o Ctrl + /	Cmd + /
Esqueleto HTML con Emmet	Escribe ! y pulsa Tab	Igual

10.2. El navegador y sus herramientas de desarrollo ★

Se abren con **F12** o **Ctrl + Mayús + I** (**Cmd + Opción + I** en macOS). Los paneles principales son:

Panel	Para qué sirve
Elements (Inspector)	Ver y modificar en vivo el DOM y los estilos
Console	Ver mensajes y errores, y ejecutar JavaScript
Sources (Depurador)	Ver el código, poner puntos de ruptura y ejecutar paso a paso (UT2)
Network (Red)	Ver cada petición y su respuesta, tiempos y tamaños
Application (Almacenamiento)	Consultar <i>cookies</i> , <i>localStorage</i> y otros almacenes (UT3)

10.3. Node.js y npm ★

Node.js ejecuta JavaScript fuera del navegador. En este módulo no lo usaremos para programar servidores (eso se ve en otros módulos), sino porque **las herramientas del desarrollo web moderno están escritas en JavaScript** y necesitan Node para funcionar: Angular CLI, ESLint, TypeScript, Vite...

npm es su gestor de paquetes: descarga librerías y herramientas del registro público npmjs.com. Tres ideas básicas:

- **package.json** describe el proyecto y sus dependencias.
- **node_modules/** es la carpeta donde se instalan. Puede ocupar cientos de megas y **nunca se sube al repositorio**: se regenera con **npm install**.
- **npm** ejecuta una herramienta sin instalarla de forma global.

10.4. Uso responsable de asistentes de IA ★

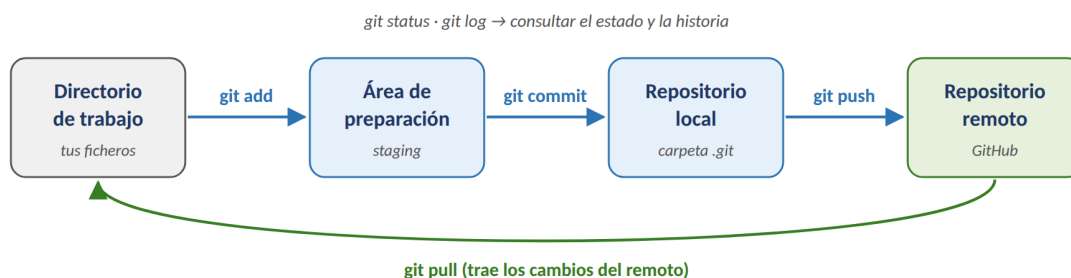
VS Code y otras herramientas incluyen asistentes de inteligencia artificial que sugieren o generan código. Pueden ser útiles, pero en esta etapa tu objetivo es **aprender a razonar el código**, no solo obtenerlo. Sigue las normas de uso que establezca tu profesorado para cada actividad y aplica siempre una regla: **no entregues nada que no seas capaz de explicar línea a línea**.

11. Git y GitHub en lo esencial

Es probable que ya conozcas Git del curso anterior; aquí repasamos lo imprescindible para trabajar desde el primer día.

11.1. Conceptos ★

- **Repositorio:** la carpeta del proyecto más su historia completa (guardada en la carpeta oculta `.git`).
- **Commit:** una «foto» del proyecto en un momento dado, con un mensaje que explica qué cambió.
- **Área de preparación** (*staging*): donde eliges qué cambios formarán parte del siguiente commit.
- **Rama:** una línea de trabajo independiente. La principal se llama `main`.
- **Remoto:** una copia del repositorio en un servidor, en nuestro caso **GitHub**.



Flujo básico de Git: los cambios pasan del directorio de trabajo al área de preparación (git add), de ahí al repositorio local (git commit) y al remoto en GitHub (git push). git pull trae los cambios del remoto.

11.2. Flujo básico ★

```
git status          # ¿Qué ha cambiado?
git add .           # Preparar todos los cambios
git commit -m "Añade página inicial del proyecto"
git push            # Subir los commits a GitHub
git log --oneline   # Ver la historia resumida
```

Todo esto también se puede hacer desde el panel **Control de código fuente** de VS Code (icono de ramas en la barra lateral).

11.3. Buenas prácticas ★

- Haz **commits pequeños y frecuentes**: uno por cada cambio con sentido propio.
- Escribe **mensajes claros**, en imperativo y que expliquen qué hace el cambio: «Añade formulario de alta», no «cambios» ni «asdf».
- Usa un fichero `.gitignore` para excluir lo que no debe subirse: `node_modules/`, ficheros del sistema (`.DS_Store`, `Thumbs.db`), claves o configuraciones locales.

- **Nunca subas contraseñas ni claves de API** al repositorio. Si ocurre por error, cámbialas: borrarlas en un commit posterior no las elimina de la historia.

12. Resumen

- Una aplicación web se reparte entre **cliente** (navegador) y **servidor**, que se comunican mediante **peticiones y respuestas HTTP**.
- El código de **servidor** se ejecuta antes de responder y está oculto; el de **cliente** se descarga, se ejecuta en el navegador y es visible y modificable. Por eso **nunca hay que confiar en el cliente**.
- En una **MPA** cada navegación descarga un documento nuevo; en una **SPA** se descarga la aplicación una vez y después solo se intercambian datos.
- El navegador es una plataforma de ejecución con motor de renderizado, motor de JavaScript, red y almacenamiento, que ejecuta el código dentro de un **entorno aislado** con limitaciones de seguridad.
- **HTML** estructura, **CSS** presenta y **JavaScript** da comportamiento. **TypeScript** añade tipos a JavaScript y se transpila a él.
- Los **lenguajes de guiones** ganan en agilidad y portabilidad, y pierden en detección temprana de errores y en control del entorno.
- La forma recomendada de integrar JavaScript es un **fichero externo** con **defer** o **type="module"**, servido desde un **servidor local**.
- Una **librería** la llamas tú; un **framework** llama a tu código. Angular es un framework completo basado en TypeScript.
- Tu entorno de trabajo es **VS Code + navegador con DevTools + Node.js + Git/GitHub**, y más adelante **Angular CLI**.

13. Glosario

Término	Definición
Cliente	Programa que solicita recursos a un servidor; en la web, el navegador.
Servidor	Programa que atiende peticiones y devuelve respuestas.
HTTP / HTTPS	Protocolo de petición-respuesta de la web; HTTPS es su versión cifrada.
URL	Dirección que identifica un recurso en la web.
Código de estado	Número de la respuesta HTTP que indica el resultado (200, 404, 500...).
MPA	Aplicación multipágina: cada navegación carga un documento nuevo.

Término	Definición
SPA	Aplicación de página única: la interfaz cambia sin recargar la página.
Renderizar	Construir y dibujar la interfaz a partir del código y los datos.
DOM	Representación en forma de árbol de objetos del documento HTML cargado.
Motor de JavaScript	Componente del navegador que ejecuta JavaScript (V8, SpiderMonkey, JavaScriptCore).
JIT	Compilación <i>just-in-time</i> : se traduce a código máquina durante la ejecución.
Web API	Interfaz que el navegador ofrece a JavaScript (DOM, fetch , almacenamiento...).
Sandbox	Entorno aislado que limita lo que puede hacer el código.
Mismo origen	Regla de seguridad que restringe el acceso entre distintos esquemas, dominios o puertos.
ECMAScript	Especificación estándar en la que se basa JavaScript.
Transpilar	Traducir código de un lenguaje o versión a otro de nivel similar (TypeScript → JavaScript).
Polyfill	Código que implementa una funcionalidad que un navegador no ofrece.
Lenguaje de guiones	Lenguaje que se ejecuta dentro de un programa anfitrión, sin compilación previa explícita.
Librería	Código reutilizable que tu programa llama cuando lo necesita.
Framework	Marco que define la estructura de la aplicación y llama a tu código (inversión de control).
Node.js	Entorno para ejecutar JavaScript fuera del navegador.
npm	Gestor de paquetes de Node.js.
Repositorio	Proyecto junto con su historia de cambios en Git.
Commit	Registro de un conjunto de cambios en el repositorio.

14. Actividades de la unidad

Las actividades se entregan según las indicaciones de tu profesor o profesora. A partir de la actividad A6, todo lo que hagas quedará además registrado en tus repositorios de GitHub.

A1. Anatomía de una petición (CE a, b)

Objetivo: observar el modelo cliente-servidor en funcionamiento.

1. Abre en el navegador la web que te indique el profesorado, abre las DevTools (F12) y ve al panel **Network**.
2. Marca **Disable cache** y recarga la página con **Ctrl + Mayús + R** (**Cmd + Mayús + R** en macOS).
3. Selecciona la **primera petición** (el documento) y anota: URL, método, código de estado y valor de la cabecera **Content-Type**.
4. En la barra inferior del panel, anota el número de peticiones, los datos transferidos y los tiempos de **DOMContentLoaded** y **Load**.
5. Usa los filtros (**Doc, CSS, JS, Img, Fetch/XHR**) y anota cuántas peticiones hay de cada tipo.
6. Compara el código fuente original (**Ctrl + U**; **Cmd + Opción + U** en macOS) con el panel **Elements**. ¿Son iguales? ¿Qué parte crees que se generó en el servidor y cuál en el cliente?

Entrega: tabla con los datos y respuesta razonada a la pregunta 6 (máximo 10 líneas).

A2. ¿MPA o SPA? (CE a, b, c)

Objetivo: clasificar aplicaciones reales y reconocer las tecnologías que usan.

Analiza las webs que te indique el profesorado más una de tu elección. En cada una:

1. Con el panel **Network** abierto (filtro **Doc**), navega por tres secciones distintas. ¿Aparece un documento nuevo en cada navegación?
2. Compara el código fuente (**Ctrl + U**; **Cmd + Opción + U** en macOS) con **Elements**. ¿El HTML inicial contiene el contenido o está casi vacío?
3. Desactiva JavaScript y recarga. En Chrome o Edge: con las DevTools abiertas, pulsa **Ctrl + Mayús + P** (**Cmd + Mayús + P** en macOS) y escribe *Disable JavaScript*. ¿Qué queda de la web? Vuelve a activarlo al terminar.
4. Intenta identificar la tecnología: en **Elements**, busca atributos como **ng-version** (Angular) o **data-v-...** (Vue), o scripts cuya ruta empiece por **/_next/** (Next.js, basado en React). También puedes usar una extensión como Wappalyzer.

Entrega: una tabla con una fila por web (MPA/SPA, evidencias, funciona sin JavaScript, tecnología detectada) y una conclusión: ¿qué tipo de web encaja con cada modelo?

A3. Lenguajes del cliente y programación de guiones (CE c, d)

Objetivo: caracterizar los lenguajes del entorno cliente y valorar el enfoque de guiones.

1. Completa una tabla comparativa de **JavaScript, TypeScript, Java y WebAssembly** con estas filas: dónde se ejecuta, si se compila y a qué, tipo de tipado, si el navegador lo entiende directamente y usos típicos.
2. Para cada ventaja y cada desventaja de la programación de guiones del apartado 7.3, escribe un **ejemplo concreto** que la ilustre.
3. Debate en grupo: **¿debería una web funcionar sin JavaScript?** Distingue entre un periódico, una tienda en línea y un editor de imágenes. Anota la conclusión del grupo.

A4. Laboratorio de integración de scripts (CE b, e)

Objetivo: comprobar experimentalmente cómo se integra y se carga JavaScript.

Crea la carpeta `ut01/a4-laboratorio` con la página del apartado 8.4 y ábrela con **Live Server**. Después, realiza estos experimentos y anota **qué ocurre y por qué**:

1. Quita `defer` del `<script>` del `<head>` y recarga. Observa la consola. ¿Qué error aparece?
2. Vuelve a poner `defer`. ¿Se soluciona? Explica la diferencia con ayuda del diagrama del apartado 8.2.
3. Mueve el script, sin `defer`, al final del `<body>`. ¿Funciona? ¿Qué inconveniente tiene esta opción?
4. Cambia `defer` por `type="module"`. Funciona con Live Server; ahora abre el fichero con doble clic (`file://`). ¿Qué dice la consola?
5. Desactiva JavaScript y recarga. ¿Qué ves? ¿Qué papel cumple `<noscript>`?
6. Añade un botón con `onclick="alert('Hola')"` y explica por qué esta forma de integración no es una buena práctica.
7. **(Opcional)** Crea dos scripts, `a.js` y `b.js`, que escriban su nombre en la consola. Cárgalos con `async` y después con `defer`, simulando una red lenta en **Network (Slow 4G)**. ¿Se respeta siempre el orden?

Entrega: los ficheros del laboratorio y un fichero `NOTAS.md` con tus observaciones.

A5. Monta tu entorno de trabajo (CE f)

Objetivo: instalar, configurar y verificar las herramientas del módulo.

1. Sigue la **Guía del entorno de trabajo** para tu sistema operativo.
2. Ejecuta la **comprobación del entorno** que aparece al final de la guía.
3. Redacta el fichero `ENTORNO.md` con: sistema operativo, versión de cada herramienta, extensiones instaladas en VS Code, una captura de tu página funcionando con Live Server y los problemas que hayas encontrado y cómo los resolviste.

A6. Tu repositorio de ejercicios (CE f)

Objetivo: poner bajo control de versiones tu trabajo de clase.

1. Configura Git con tu nombre y tu correo (lo tienes en la guía).
2. Crea la carpeta **dew-ejercicios** con la subcarpeta **ut01** y guarda en ella el **ENTORNO.md** de A5. Añade en la raíz un **README.md** breve, un **.gitignore** y un **.gitattributes** (tienes los modelos en el apartado 10.1 de la guía).
3. Inicializa el repositorio y haz tu primer commit. Cuando termines A4, añade sus ficheros en **ut01** con nuevos commits: al acabar la unidad debe haber **al menos tres commits** con mensajes significativos.
4. Publícalo en GitHub desde VS Code (**Publish to GitHub**) como repositorio privado y añade a tu profesor o profesora como colaborador.

A7. La ficha de tu proyecto (CE c, e, f)

Objetivo: definir tu proyecto del curso y dejarlo en marcha.

1. Crea un segundo repositorio, **dew-proyecto**, siguiendo los mismos pasos que en A6.
2. Escribe su **README.md** con esta estructura:
 - **Nombre** y descripción en dos o tres frases.
 - **Público:** quién la usaría y para qué.
 - **Entidades:** la principal (con sus propiedades y tipos) y la relacionada.
 - **Funcionalidades:** cómo cumplirás cada requisito, de R1 a R10.
 - **Servicio web:** una API pública candidata o «API simulada».
 - **Boceto** de la pantalla principal (foto de un dibujo a mano o captura de una herramienta como Excalidraw).
3. Crea la página inicial adaptando la del apartado 8.4 con el nombre de tu proyecto y comprueba que funciona con Live Server.
4. Haz commit y push. Tu profesor o profesora revisará el tema y te dará el visto bueno o te propondrá ajustes.

A8. Elegir herramienta (CE c, f)

Objetivo: evaluar frameworks con criterios objetivos.

Por parejas y consultando las **webs oficiales** de Angular, React y Vue:

1. Anota para cada uno: versión actual, licencia, lenguaje, herramienta oficial para crear proyectos y si incluye de serie rutas, formularios y cliente HTTP.
2. Busca cada paquete en **npmjs.com** (**@angular/core**, **react**, **vue**) y anota sus descargas semanales.
3. Recomienda uno para cada escenario y justifícalo en dos o tres líneas:
 - La intranet de una aseguradora que mantendrá un equipo de 20 personas durante diez años.

- Un pequeño widget interactivo dentro de una web ya existente.
- La web de un restaurante con carta y formulario de reservas.

15. Autoevaluación

1. ¿Qué parte de la URL `https://ejemplo.com/tienda?pagina=2#ofertas` no se envía nunca al servidor?
2. Una tienda online valida el precio final de una compra solo con JavaScript en el cliente. ¿Qué problema tiene?
3. Al navegar por una web, en el panel Network solo aparecen peticiones Fetch/XHR y ningún documento nuevo. ¿Qué tipo de aplicación es?
4. Cita dos componentes de un navegador y el nombre del motor de JavaScript de Chrome.
5. Indica dos limitaciones del código JavaScript que se ejecuta en el navegador.
6. ¿Por qué el navegador no puede ejecutar directamente un fichero TypeScript?
7. Cita una ventaja y una desventaja de los lenguajes de guiones frente a Java.
8. Tienes `<script src="app.js"></script>` en el `<head>` y el script accede a un elemento del `<body>`. ¿Qué ocurre y cómo se soluciona?
9. ¿Qué diferencia hay entre `async` y `defer`?
10. Explica con tus palabras la diferencia entre una librería y un framework.

16. Recursos

- **MDN Web Docs** (en español): <https://developer.mozilla.org/es/> — la referencia de HTML, CSS, JavaScript y las Web APIs.
- **javascript.info** (versión en español): <https://es.javascript.info/> — tutorial moderno y muy completo.
- **Documentación oficial de Angular**: <https://angular.dev/> — incluye tutoriales interactivos.
- **Can I use**: <https://caniuse.com/> — compatibilidad entre navegadores.
- **web.dev · Baseline**: <https://web.dev/baseline> — qué funciones son seguras de usar hoy.
- **Documentación de VS Code**: <https://code.visualstudio.com/docs>
- **Pro Git** (libro gratuito, en español): <https://git-scm.com/book/es/v2>
- **Chrome DevTools**: <https://developer.chrome.com/docs/devtools>